
JavaScript Tutorial

Ref: <https://www.w3schools.com/> and
other internet based tutorials

Looping

- while Loops

```
<script>  
counter=0  
while (counter < 5)  
{  
  document.write("Counter: " + counter + "<br>")  
  ++counter  
}  
</script>
```

Looping

- do...while Loops

```
<script>  
count = 1  
do  
{  
document.write(count + " times 7 is " + count * 7 + "<br>")  
} while (++count <= 7)  
</script>
```

Looping

- for Loops

```
<script>
```

```
for (count = 1 ; count <= 7 ; ++count)
```

```
{
```

```
document.write(count + "times 7 is " + count * 7 + "<br>");
```

```
}
```

```
</script>
```

JavaScript Functions

- A JavaScript function is a block of code designed to perform a particular task.
- A JavaScript function is executed when "something" invokes it (calls it).
- Example

```
function myFunction(p1, p2) {  
    return p1 * p2; // The function returns the product of p1 and p2  
}
```

JavaScript Function Syntax

- A JavaScript function is defined with the function keyword, followed by a name, followed by parentheses ().
- Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).
- The parentheses may include parameter names separated by commas:

```
function name(parameter1, parameter2, parameter3)
{
    // code to be executed
}
```

Function Invocation

- The code inside the function will execute when "something" invokes (calls) the function:
 - ❑ When an event occurs (when a user clicks a button)
 - ❑ When it is invoked (called) from JavaScript code
 - ❑ Automatically (self invoked)
-

Automatically (self invoked) Functions

- Function expressions can be made "self-invoking".
- A self-invoking expression is invoked (started) automatically, without being called.
- Function expressions will execute automatically if the expression is followed by ().

```
<script>
(function ()
{
    document.getElementById("demo").innerHTML = "Hello! I called
myself";
})();
</script>
```


Function Return

- When JavaScript reaches a return statement, the function will stop executing.
- If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.
- Functions often compute a return value. The return value is "returned" back to the "caller":
- Example: Calculate the product of two numbers, and return the result:

var x = myFunction(4, 3); // Function is called, return value will be stored in x

```
function myFunction(a, b) {  
    return a * b;           // Function returns the product of a and b  
}
```

() Operator in function

- Function name only (like `sum`) refers to the function object, and `sum()` refers to the function result.
- Accessing a function without `()` will return the function definition instead of the function result:
- Example

```
function toCelsius(fahrenheit) {  
    return (5/9) * (fahrenheit-32);  
}
```

```
document.getElementById("demo").innerHTML = toCelsius;
```

Functions Used as Variable Values

- Functions can be used the same way as you use variables, in all types of formulas, assignments, and calculations.
- Example: using a variable to store the return value of a function:
var x = toCelsius(77);
var text = "The temperature is " + x + " Celsius";
- You can use the function directly, as a variable value:
var text = "The temperature is " + toCelsius(77) + " Celsius";

JavaScript Classes

- Class Definition
- Use the keyword `class` to create a class, and always add a constructor method.
- The constructor method is called each time the class object is initialized.
- Example :A simple class definition for a class named "Car":

```
class Car {  
  constructor(brand) {  
    this.carname = brand;  
  }  
}  
  
mycar = new Car("Ford");
```

Getters and Setters

- Classes also allows you to use getters and setters.
- It can be smart to use getters and setters for your properties, especially if you want to do something special with the value before returning them, or before you set them.
- To add getters and setters in the class, use the `get` and `set` keywords.
- **Note:** even if the getter is a method, you do not use parentheses when you want to get the property value.

Example

```
class Car {  
    constructor(brand) {  
        this.carname = brand;  
    }  
    get cnam() {  
        return this.carname;  
    }  
    set cnam(x) {  
        this.carname = x;  
    }  
}  
  
mycar = new Car("Ford");  
  
document.getElementById("demo").innerHTML = mycar.cnam;
```

Example: Use a setter to change the carname to "Volvo":

```
class Car {  
    constructor(brand) {  
        this._carname = brand;  
    }  
    get carname() {  
        return this._carname;  
    }  
    set carname(x) {  
        this._carname = x;  
    }  
}
```

```
mycar = new Car("Ford");
```

```
mycar.carname = "Volvo";
```

```
document.getElementById("demo").innerHTML =  
mycar.carname;
```